ROS 2 with Python on WSL — Integrated Guide: Single-Node and Multi-Node Projects

This is the single PDF version. Each project contains its own explanation, single-node version, multi-node version, setup, dependencies, code, build instructions, run instructions, and completion check. Installation, concepts, setup, code, build commands, run commands, and completion checks are kept together.

Part 1 — Complete beginner installation

ROS 2 with Python on WSL

Absolute Beginner Guide: Every Step from Windows to the First Project

This guide assumes the learner has **never used WSL, Ubuntu, Linux, or ROS 2** before.

The computer shown in the image is suitable for learning:

- Windows 11
- 12th Gen Intel Core i5-1235U
- 16 GB RAM
- Intel Iris Xe graphics
- Approximately 114 GB free storage

The guide uses:

- WSL 2
- Ubuntu 24.04
- ROS 2 Jazzy
- Python 3

Part A — Important words before starting

What is Windows Terminal?

Windows Terminal is a Windows application that can open PowerShell, Command Prompt, and Ubuntu tabs. We will use it to install WSL.

What is Administrator mode?

Administrator mode gives Windows permission to install system components. It is needed only for some Windows installation commands.

> The learner does **not** need to use a special “Administrator Ubuntu” terminal. Administrator mode is only needed when running the initial Windows WSL installation command.

What is WSL?

WSL means **Windows Subsystem for Linux**. It allows Ubuntu and Linux commands to run inside Windows.

What is Ubuntu?

Ubuntu is the Linux system that will run inside WSL. ROS 2 will be installed inside Ubuntu, not directly inside the normal Windows desktop.

What is ROS 2?

ROS 2 is a software framework for building robot applications. It provides communication tools, messages, packages, simulation tools, and hardware support.

What is Python `rclpy`?

`rclpy` is the Python library used to write ROS 2 programs.

Part B — Before typing anything

Step B1 — Save important work

Ask the learner to save documents and close applications. The computer may need to restart during WSL installation.

Step B2 — Check free storage

The image shows approximately 114 GB free, which is enough for this learning setup. Keep at least 30 GB free if possible because ROS 2, simulation files, and projects use storage.

Step B3 — Check the Windows version

1. Click the **Start** button.
2. Type `About your PC`.
3. Open **About your PC**.
4. Look for **Windows specifications**.
5. Confirm that the system is Windows 11 or a supported recent Windows 10 version.

If the computer is Windows 11, continue.

Step B4 — Check virtualization if installation fails

Most modern computers support virtualization. If WSL reports a virtualization error:

1. Restart the computer.
2. Enter BIOS/UEFI using the key shown during startup. Common keys are `F2`, `Delete`, or `Esc`.
3. Find a setting named **Intel Virtualization Technology**, **VT-x**, or **Virtualization**.
4. Enable it.
5. Save and restart Windows.

Do not change unrelated BIOS settings.

Part C — Open PowerShell as Administrator

This is the first Windows step.

Method 1 — Start menu

1. Click the **Windows Start button**.
2. Type **PowerShell**.
3. Find **Windows PowerShell** or **PowerShell** in the results.
4. Right-click it.
5. Select **Run as administrator**.
6. Windows may show a blue or gray permission window asking:

> Do you want to allow this app to make changes to your device?

1. Click **Yes**.

A window opens. The title should contain **Administrator**.

Method 2 — Windows Terminal

1. Click **Start**.
2. Type **Windows Terminal**.
3. Right-click **Windows Terminal**.
4. Select **Run as administrator**.
5. Click **Yes** in the permission window.
6. If Terminal opens PowerShell, that is correct.

> The learner does not have to open “Administrator Command Prompt” specifically.
> **Administrator PowerShell** or **Administrator Windows Terminal** is fine.

Part D — Install WSL and Ubuntu

Step D1 — Type the installation command

In the Administrator PowerShell window, type or paste this command:

```
wsl --install
```

Then press **Enter**.

What this command does:

- Enables the Windows Subsystem for Linux.
- Enables the virtual machine component needed by WSL 2.
- Installs the default Ubuntu distribution.
- Downloads the required Linux components.

The download may take several minutes. Do not close the window while it is working.

Step D2 — Restart Windows

If Windows asks you to restart, save your work and restart the computer.

After the restart, Ubuntu may open automatically. If it does not, continue to the next step.

Step D3 — If `wsl --install` does not work

Open **PowerShell as Administrator** again and type:

```
wsl --list --online
```

This displays available Linux distributions. Look for `Ubuntu-24.04` or `Ubuntu`.

Install Ubuntu 24.04 explicitly:

```
wsl --install -d Ubuntu-24.04
```

If that exact name is not listed, use the name displayed by `wsl --list --online`, for example:

```
wsl --install -d Ubuntu
```

If the download stays at 0.0%, try:

```
wsl --install --web-download -d Ubuntu-24.04
```

Use the exact distribution name shown on the computer.

Step D4 — Open Ubuntu for the first time

After installation:

1. Click **Start**.
2. Type **Ubuntu**.
3. Click the Ubuntu application.

The first launch may display:

```
Installing, this may take a few minutes...
```

Wait until it finishes.

Step D5 — Create a Linux username

Ubuntu asks:

```
Enter new UNIX username:
```

Type a simple lowercase username, for example:

```
robotstudent
```

Press **Enter**.

Use lowercase letters and numbers. Do not use spaces.

Step D6 — Create a Linux password

Ubuntu asks:

```
New password:
```

Type a password and press **Enter**.

The screen will show no stars and no letters while typing. This is normal Linux security behavior.

Ubuntu asks you to type the password again. Type it and press **Enter**.

> This Linux password is separate from the normal Windows password. Do not forget it because `sudo` commands will use it.

When complete, the terminal shows a prompt similar to:

```
robotstudent@computer-name:~$
```

The `$` means the terminal is ready for a normal user command.

Part E — Confirm that WSL 2 is working

Close the Ubuntu window for a moment.

Open **PowerShell as Administrator** again:

1. Click Start.

2. Search for PowerShell.
3. Right-click PowerShell.
4. Choose **Run as administrator**.
5. Click **Yes**.

Run:

```
wsl --list --verbose
```

You should see a table similar to:

```
  NAME            STATE           VERSION
* Ubuntu-24.04    Running         2
```

The important value is the final number: **2**.

If the value is `1`, change it with:

```
wsl --set-version Ubuntu-24.04 2
```

If the displayed name is `Ubuntu`, use:

```
wsl --set-version Ubuntu 2
```

Set WSL 2 as the default for future installations:

```
wsl --set-default-version 2
```

Close PowerShell. From this point, most commands are run inside the **Ubuntu** window, not PowerShell.

Part F — Learn how to identify the correct terminal

PowerShell prompt

A PowerShell prompt may look like:

```
PS C:\Users\Name>
```

Use PowerShell for Windows commands such as:

```
wsl --list --verbose
wsl --update
wsl --shutdown
```

Ubuntu prompt

An Ubuntu prompt may look like:

```
robotstudent@computer-name:~$
```

Use Ubuntu for Linux and ROS commands such as:

```
sudo apt update
ros2 --help
colcon build
```

> Do not run `sudo apt` or `ros2` commands in PowerShell.

Part G — Update Ubuntu

Open Ubuntu from the Start menu.

Run one command at a time. After typing each command, press **Enter**.

```
sudo apt update
```

Ubuntu asks for the Linux password. Type the password created earlier and press **Enter**.

The password remains invisible while typing. Wait until the command finishes.

Then run:

```
sudo apt upgrade -y
```

What these commands do:

- `sudo` requests administrator permission inside Ubuntu.
- `apt` is Ubuntu's package manager.
- `update` downloads the latest package list.
- `upgrade` installs available updates.
- `-y` automatically answers yes to the installation question.

Part H — Install ROS 2 Jazzy

Run the following commands inside Ubuntu.

Step H1 — Enable the Universe repository

```
sudo apt install software-properties-common -y
```

Then:

```
sudo add-apt-repository universe
```

If it asks for confirmation, press **Enter**.

Step H2 — Install curl

```
sudo apt update
sudo apt install curl -y
```

`curl` downloads the official ROS repository key.

Step H3 — Add the official ROS repository key

Copy the entire command, including the backslash line continuation:

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros
  -o /usr/share/keyrings/ros-archive-keyring.gpg
```

The command may produce no visible output. That is acceptable.

Step H4 — Add the ROS 2 package repository

Copy the complete command:

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings
  sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

This command automatically detects the Ubuntu release name and adds the correct ROS package source.

Step H5 — Install ROS 2 Desktop

```
sudo apt update
sudo apt install ros-jazzy-desktop ros-dev-tools -y
```

This can take several minutes and download a large amount of data. Do not close the terminal.

The desktop installation includes the tools useful for learning, including demonstrations and RViz.

Part I — Configure ROS 2 so commands work

Step I1 — Test ROS 2 in the current terminal

```
source /opt/ros/jazzy/setup.bash
```

Now run:

```
ros2 --help
```

If ROS 2 is installed correctly, help text appears.

Step I2 — Make ROS 2 work in every new terminal

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
```

Reload the configuration:

```
source ~/.bashrc
```

Step I3 — Check Python and rclpy

```
python3 --version
```

Then:

```
python3 -c "import rclpy; print('rclpy is installed correctly')"
```

Expected output:

```
rclpy is installed correctly
```

Part J — Test ROS 2 before writing code

Step J1 — Start the sample publisher

Open a second Ubuntu terminal. Keep the first Ubuntu terminal open.

In the second terminal, run:

```
ros2 run demo_nodes_cpp talker
```

You should see messages being published.

Step J2 — Start the sample Python subscriber

Open a third Ubuntu terminal and run:

```
ros2 run demo_nodes_py listener
```

You should see messages similar to:

```
I heard: [Hello World: 1]
```

This proves that two ROS 2 nodes can communicate.

Stop both programs with:

```
Ctrl+C
```

`Ctrl+C` means hold the **Ctrl** key and press the **C** key.

Part K — Create the Python ROS 2 workspace

Use one Ubuntu terminal:

```
mkdir -p ~/ros2_ws/src
```

Explanation:

- `mkdir` creates a directory.

- `-p` creates parent directories when necessary.
- `~` means the current Linux home directory.
- `ros2_ws` is the workspace name.
- `src` will contain source packages.

Move into the workspace:

```
cd ~/ros2_ws
```

Build it:

```
colcon build
```

Source it:

```
source install/setup.bash
```

Automatically source it in new terminals:

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Check the current folder:

```
pwd
```

Expected form:

```
/home/robotstudent/ros2_ws
```

Part L — First beginner project: Hello Python node

Step L1 — Create the package

```
cd ~/ros2_ws/src
```

Then:

```
ros2 pkg create --build-type ament_python \
  --license Apache-2.0 \
  hello_robot
```

This creates a Python ROS 2 package.

Step L2 — Create the Python file

```
nano ~/ros2_ws/src/hello_robot/hello_robot/hello_node.py
```

If `nano` opens a blank editor, paste this code:

```python
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node


class HelloNode(Node):
    def __init__(self):
        # This is the node name in the ROS 2 graph.
        super().__init__('hello_node')

        # Call timer_callback once every second.
        self.timer = self.create_timer(1.0, self.timer_callback)

        self.get_logger().info('Hello node started')

    def timer_callback(self):
        # This message appears once every second.
        self.get_logger().info('Hello from Python ROS 2')


def main(args=None):
    # Start the ROS 2 Python library.
    rclpy.init(args=args)

    node = HelloNode()

    try:
        # Keep the node running and process callbacks.
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        # Cleanly stop the node.
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Save in nano:

1. Press **Ctrl+O**.
2. Press **Enter** to confirm the filename.
3. Press **Ctrl+X** to exit.

Step L3 — Add the run command

Open `setup.py`:

```
nano ~/ros2_ws/src/hello_robot/setup.py
```

Find the `entry_points` section. Replace or add it so it contains:

```
entry_points={
    'console_scripts': [
        'hello = hello_robot.hello_node:main',
    ],
},
```

Save with **Ctrl+O**, press **Enter**, and exit with **Ctrl+X**.

Step L4 — Build the package

```
cd ~/ros2_ws
colcon build --symlink-install
```

If the build finishes without an error, run:

```
source install/setup.bash
```

Step L5 — Run the node

```
ros2 run hello_robot hello
```

Expected output appears every second:

```
[INFO] [hello_node]: Hello from Python ROS 2
```

Stop with **Ctrl+C**.

Part M — Troubleshooting for a complete beginner

Problem: `wsl` is not recognized

1. Close PowerShell.
2. Open Start.
3. Search for **Windows Terminal**.
4. Right-click it.
5. Select **Run as administrator**.
6. Try `wsl --install` again.

If it still fails, restart Windows and try once more.

Problem: the terminal is not Administrator

Look at the title bar. It should contain `Administrator` when doing the Windows installation step.

If it does not:

1. Close the terminal.
2. Right-click PowerShell or Windows Terminal.
3. Select **Run as administrator**.
4. Click **Yes**.

Problem: Ubuntu asks for a password but nothing appears

This is normal. Linux does not display password characters. Type the password and press **Enter**.

Problem: `ros2: command not found`

Inside Ubuntu, run:

```
source /opt/ros/jazzy/setup.bash
```

Then:

```
ros2 --help
```

If it works, add the source line permanently:

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Problem: `colcon: command not found`

Run:

```
sudo apt update
sudo apt install ros-dev-tools -y
```

Then:

```
source /opt/ros/jazzy/setup.bash
```

Problem: `Package 'hello_robot' not found`

Run all three commands:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 pkg list | grep hello_robot
```

If the package is not listed, check that it is located here:

```
ls ~/ros2_ws/src/hello_robot
```

Problem: Python executable is not found

Check `setup.py`. It must contain:

```
'hello = hello_robot.hello_node:main',
```

Then rebuild:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
```

Problem: RViz window does not appear

From PowerShell, not Ubuntu:

```
wsl --update
wsl --shutdown
```

Open Ubuntu again and try:

```
rviz2
```

If graphics are slow, begin with terminal projects and `turtlesim`. Use small Gazebo worlds later.

Problem: a command seems frozen

Wait a few minutes when installing ROS 2. Package installation can take time.

If a normal program is running and needs to stop, press:

```
Ctrl+C
```

Do not press random keys while `apt` is installing packages.

Part N — What to learn after the first node

Complete projects in this order:

1. **Hello timer node** — node, timer, logging, and `spin`.
2. **Temperature publisher/subscriber** — topics, messages, and callbacks.
3. **Addition or counter service** — request/response communication.
4. **Parameters and launch file** — configurable systems and repeatable startup.
5. **Turtlesim square controller** — velocity commands and robot motion.
6. **Custom message project** — create a `.msg` interface.
7. **LaserScan safety node** — sensor processing and obstacle stopping.
8. **URDF and RViz robot** — robot links, joints, and coordinate frames.
9. **Gazebo simulated robot** — robot simulation and sensors.
10. **TF2 and odometry** — coordinate transforms and robot position.
11. **Python action server/client** — goals, feedback, results, and cancellation.
12. **Rosbag project** — record and replay robot data.
13. **Navigation project** — send navigation goals to a simulated robot.
14. **Final capstone** — autonomous Python robot in simulation.

Do not move forward only because the code runs. For each project, the learner should be able to explain what each node does and how the nodes communicate.

Part O — The first five projects in simple words

Project 1: Hello node

One Python program prints a message every second.

Main idea: a ROS 2 node is a running program.

Project 2: Temperature system

One program publishes a temperature. Another program receives it.

Main idea: nodes communicate through topics.

Project 3: Calculator service

One program asks another program to calculate a result.

Main idea: services use request and response.

Project 4: Configurable system

The same program behaves differently when parameters change. A launch file starts the system.

Main idea: real robot systems should be configurable and repeatable.

Project 5: Turtle controller

A Python program sends velocity commands to a simulated turtle.

Main idea: ROS 2 Python code can control a simulated robot.

Part P — Rules for teaching the learner

1. Copy one command at a time.
2. Read the explanation before pressing Enter.
3. Do not use `sudo` unless the guide says to use it.
4. Do not delete folders to fix an error before reading the error.
5. Keep a notebook with commands and meanings.
6. Take a screenshot when a project works.
7. Save each project in Git.
8. Use simulation before connecting a real robot.
9. Never test unrestricted velocity commands on a real robot.
10. Ask what terminal is being used: PowerShell or Ubuntu.

References

Part 2 — Projects with setup and code together

Project 1 — Hello timer node

ROS concepts: node, timer, logger, spin, shutdown.

Project 1 explanation — single node and multi-node version

Single-node version: one Python node owns the timer and prints a message. This teaches the basic ROS 2 node lifecycle without communication between nodes.

Multi-node version: create a second `hello_listener` node and publish the text on a `/hello` topic. The first node becomes a publisher and the second node becomes a subscriber. This teaches that ROS 2 nodes are separate programs that communicate through topics.

What to understand: `rclpy.init()` starts ROS 2, `Node` creates the program in the ROS graph, `create_timer()` creates repeated work, `spin()` processes callbacks, and `shutdown()` closes ROS 2 cleanly.

Project 1 setup

Run inside Ubuntu:

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python ros2_projects
mkdir -p ~/ros2_ws/src/ros2_projects/ros2_projects
```

Create `ros2_projects/ros2_projects/project1_hello.py`. Add this executable to `setup.py`:

```
'hello = ros2_projects.project1_hello:main',
```

Build and run after copying the code below:

```
cd ~/ros2_ws
colcon build --symlink-install
source install/setup.bash
ros2 run ros2_projects hello
```

File: `project1_hello.py`

```
import rclpy
from rclpy.node import Node


class HelloNode(Node):
    def __init__(self):
        super().__init__('hello_node')
        # ROS calls timer_callback once every second.
        self.timer = self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        self.get_logger().info('Hello from my first ROS 2 Python node')


def main(args=None):
    rclpy.init(args=args)
    node = HelloNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Run:

```
ros2 run ros2_projects hello
```

Project 2 — Temperature publisher and subscriber

ROS concepts: topic, publisher, subscriber, message, callback, queue depth.

Project 2 explanation — two communicating nodes

Single-node version: one node generates a temperature and checks its own value. This is useful for learning timers, parameters, and conditional logic.

Multi-node version: use the publisher and subscriber supplied below. The publisher creates `Float32` messages on `/temperature`; the subscriber receives them in a callback and reports normal or high temperature. This is the main beginner example of ROS 2 communication.

What to understand: the publisher and subscriber do not call each other directly. The topic name and message type connect them through the ROS 2 graph.

Project 2 setup

Add these files to the existing `ros2_projects` package:

```
ros2_projects/ros2_projects/project2_temperature_publisher.py
ros2_projects/ros2_projects/project2_temperature_subscriber.py
```

Add these entries to `setup.py`:

```
'temperature_publisher = ros2_projects.project2_temperature_publisher:mai
'temperature_subscriber = ros2_projects.project2_temperature_subscriber:ma
```

The package uses `rclpy` and `std_msgs`; add `<depend>rclpy</depend>` and `<depend>std_msgs</depend>` to `package.xml`.

Publisher: `project2_temperature_publisher.py`

```
import random

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32


class TemperaturePublisher(Node):
    def __init__(self):
        super().__init__('temperature_publisher')
        self.publisher = self.create_publisher(Float32, '/temperature', 1(
        self.timer = self.create_timer(1.0, self.publish_temperature)

    def publish_temperature(self):
        message = Float32()
        message.data = random.uniform(20.0, 30.0)
        self.publisher.publish(message)
        self.get_logger().info(f'Published: {message.data:.2f} C')


def main(args=None):
    rclpy.init(args=args)
    node = TemperaturePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Subscriber: `project2_temperature_subscriber.py`

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32


class TemperatureSubscriber(Node):
    def __init__(self):
        super().__init__('temperature_subscriber')
        self.declare_parameter('warning_temperature', 28.0)
        self.limit = self.get_parameter('warning_temperature').value
        self.subscription = self.create_subscription(
            Float32,
            '/temperature',
            self.temperature_callback,
            10,
        )

    def temperature_callback(self, message):
        if message.data ⩾ self.limit:
            self.get_logger().warn(f'High temperature: {message.data:.2f}
        else:
            self.get_logger().info(f'Normal temperature: {message.data:.2


def main(args=None):
    rclpy.init(args=args)
    node = TemperatureSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Run in two terminals:

```
ros2 run ros2_projects temperature_publisher
ros2 run ros2_projects temperature_subscriber
```

Project 3 — Addition service and client

ROS concepts: service, request, response, service server, service client.

Project 3 explanation — service server and client

Single-node version: one node owns a counter and resets it through an internal Python method. This teaches state and callback logic.

Multi-node version: the service server below provides `/add_two_numbers`, while the client sends a request and receives a response. The server and client are separate nodes.

What to understand: a service is appropriate for a short request and response. The client waits for the service, sends a request, and receives a result.

Project 3 setup

Add:

```
ros2_projects/ros2_projects/project3_add_server.py
ros2_projects/ros2_projects/project3_add_client.py
```

Add both console-script entries to `setup.py` and add this dependency to `package.xml`:

```
<depend>example_interfaces</depend>
```

Build, then run the server in one terminal and the client in another.

Service server: `project3_add_server.py`

```python
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts


class AddServer(Node):
    def __init__(self):
        super().__init__('add_server')
        self.service = self.create_service(
            AddTwoInts,
            '/add_two_numbers',
            self.add_callback,
        )

    def add_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info(
            f'{request.a} + {request.b} = {response.sum}'
        )
        return response


def main(args=None):
    rclpy.init(args=args)
    node = AddServer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Service client: `project3_add_client.py`

```python
import sys

import rclpy
from example_interfaces.srv import AddTwoInts


def main(args=None):
    rclpy.init(args=args)
    node = rclpy.create_node('add_client')
    client = node.create_client(AddTwoInts, '/add_two_numbers')

    while not client.wait_for_service(timeout_sec=1.0):
        node.get_logger().info('Waiting for /add_two_numbers...')

    request = AddTwoInts.Request()
    request.a = int(sys.argv[1]) if len(sys.argv) > 1 else 2
    request.b = int(sys.argv[2]) if len(sys.argv) > 2 else 3

    future = client.call_async(request)
    rclpy.spin_until_future_complete(node, future)

    if future.result() is not None:
        node.get_logger().info(f'Result: {future.result().sum}')
    else:
        node.get_logger().error('Service call failed')

    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Add `example_interfaces` to `package.xml`. Run:

```
ros2 run ros2_projects add_server
ros2 run ros2_projects add_client 7 5
```

Project 4 — Parameters and launch file

ROS concepts: parameter, YAML configuration, launch, multiple nodes.

Project 4 explanation — configurable node and launch system

Single-node version: start `configurable_node` alone with command-line parameters or a YAML file. This teaches how a node reads configuration.

Multi-node version: use a launch file to start a publisher and a subscriber together, each with its own parameters. This teaches how real robot systems are assembled from several independent nodes.

What to understand: parameters change behavior without editing source code, while a launch file makes the complete system repeatable.

Project 4 setup

Add the Python node and create these folders:

```
mkdir -p ~/ros2_ws/src/ros2_projects/config
mkdir -p ~/ros2_ws/src/ros2_projects/launch
```

Place the YAML file in `config/project4.yaml` and the launch file in `launch/project4.launch.py`. Add the launch and config directories to `data_files` in `setup.py`, then add:

```
'configurable_node = ros2_projects.project4_parameter_node:main',
```

File: `project4_parameter_node.py`

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String


class ConfigurableNode(Node):
    def __init__(self):
        super().__init__('configurable_node')
        self.declare_parameter('robot_name', 'student_robot')
        self.declare_parameter('publish_period', 1.0)

        self.robot_name = self.get_parameter('robot_name').value
        period = self.get_parameter('publish_period').value
        self.publisher = self.create_publisher(String, '/robot_status', 1
        self.timer = self.create_timer(period, self.publish_status)

    def publish_status(self):
        message = String()
        message.data = f'{self.robot_name}: running'
        self.publisher.publish(message)
        self.get_logger().info(message.data)


def main(args=None):
    rclpy.init(args=args)
    node = ConfigurableNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

YAML file: `config/project4.yaml`

```yaml
configurable_node:
  ros__parameters:
    robot_name: "demo_robot"
    publish_period: 0.5
```

Launch file: `launch/project4.launch.py`

```python
from launch import LaunchDescription
from launch_ros.actions import Node


def generate_launch_description():
    return LaunchDescription([
        Node(
            package='ros2_projects',
            executable='configurable_node',
            name='configurable_node',
            parameters=['config/project4.yaml'],
            output='screen',
        ),
    ])
```

Run with a command-line parameter:

```
ros2 run ros2_projects configurable_node --ros-args -p publish_period:=0.1
```

Project 5 — Turtlesim square controller

ROS concepts: `Twist`, velocity, timer, state machine, safe stop.

Project 5 explanation — one controller and a two-node controller

Single-node version: the square controller alone publishes `Twist` commands to the already-running turtlesim node. The controller is one custom node; turtlesim is the robot simulator node.

Multi-node version: split the controller into a `motion_node` that publishes commands and a `pose_monitor_node` that subscribes to `/turtle1/pose` and reports progress. This teaches command and feedback nodes.

What to understand: `linear.x` controls forward movement, `angular.z` controls turning, and a zero `Twist` is required for safe stopping.

Project 5 setup

Add `project5_turtle_square.py` to the Python package and add this entry to `setup.py`:

```
'turtle_square = ros2_projects.project5_turtle_square:main',
```

Add `<depend>geometry_msgs</depend>` to `package.xml`. Start turtlesim before the controller.

File: `project5_turtle_square.py`

```python
import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node


class TurtleSquare(Node):
    def __init__(self):
        super().__init__('turtle_square')
        self.publisher = self.create_publisher(
            Twist, '/turtle1/cmd_vel', 10
        )
        self.state = 'forward'
        self.state_time = 0.0
        self.side_count = 0
        self.timer = self.create_timer(0.1, self.control_loop)

    def control_loop(self):
        command = Twist()
        self.state_time += 0.1

        if self.side_count >= 4:
            self.publisher.publish(command)  # zero velocity: stop
            self.get_logger().info('Square complete')
            self.timer.cancel()
            return

        if self.state == 'forward':
            command.linear.x = 1.0
            if self.state_time >= 2.0:
                self.state = 'turn'
                self.state_time = 0.0
        else:
            command.angular.z = 1.57
            if self.state_time >= 1.0:
                self.state = 'forward'
                self.state_time = 0.0
                self.side_count += 1

        self.publisher.publish(command)


def main(args=None):
    rclpy.init(args=args)
```

```
    node = TurtleSquare()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Run:

```
ros2 run turtlesim turtlesim_node
ros2 run ros2_projects turtle_square
```

Project 6 — Custom robot status message

ROS concepts: custom interface, message fields, interface package, topic communication.

Project 6 explanation — custom message with separate nodes

Single-node version: one node creates and prints a `RobotStatus` message locally. This teaches the fields of a custom interface.

Multi-node version: a status publisher sends `RobotStatus` on `/robot_status`, and a monitor subscriber checks battery and obstacle fields. The interface package is separate from the Python node package so other packages can reuse the message.

What to understand: a custom message is a typed contract. All nodes must use the same generated interface definition.

Project 6 setup

Create the interface package first:

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake robot_interfaces
mkdir -p robot_interfaces/msg
```

Create `robot_interfaces/msg/RobotStatus.msg` using the definition below. Build `robot_interfaces` before the Python package. Add the generated interface package as a dependency of `ros2_projects`.

Create an interface package:

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake robot_interfaces
mkdir -p robot_interfaces/msg
```

File: `robot_interfaces/msg/RobotStatus.msg`

```
string robot_name
float32 battery_percentage
bool obstacle_detected
string state
```

Add this to `robot_interfaces/CMakeLists.txt`:

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/RobotStatus.msg"
)

ament_export_dependencies(rosidl_default_runtime)
```

Add these dependencies to `robot_interfaces/package.xml`:

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

Python publisher: `project6_status_publisher.py`

```python
import rclpy
from rclpy.node import Node
from robot_interfaces.msg import RobotStatus


class StatusPublisher(Node):
    def __init__(self):
        super().__init__('status_publisher')
        self.publisher = self.create_publisher(RobotStatus, '/robot_status
        self.battery = 100.0
        self.timer = self.create_timer(1.0, self.publish_status)

    def publish_status(self):
        message = RobotStatus()
        message.robot_name = 'student_robot'
        message.battery_percentage = self.battery
        message.obstacle_detected = False
        message.state = 'ready'
        self.publisher.publish(message)
        self.get_logger().info(f'Battery: {self.battery:.1f}%')
        self.battery = max(0.0, self.battery - 1.0)


def main(args=None):
    rclpy.init(args=args)
    node = StatusPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Add the interface package as a dependency of the Python package, then build the interface package before the Python package.

Project 7 — LaserScan obstacle detector

ROS concepts: sensor topic, `LaserScan`, ranges, invalid values, safety stop.

Project 7 explanation — sensor node and safety node

Single-node version: one node subscribes to `/scan`, detects the nearest obstacle, and publishes a stop or forward command. This is the runnable code below.

Multi-node version: separate the system into a `scan_processor` node that publishes `/obstacle_distance` and a `safety_controller` node that subscribes to that topic and publishes `/cmd_vel`. This separation makes testing easier.

What to understand: sensor messages contain many readings, invalid values must be filtered, and safety behavior should override normal motion.

Project 7 setup

Add `project7_obstacle_stop.py` to the Python package. Add these dependencies to `package.xml`:

```
<depend>geometry_msgs</depend>
<depend>sensor_msgs</depend>
```

The simulator must publish `/scan` and accept `/cmd_vel`. Test in simulation before using hardware.

File: `project7_obstacle_stop.py`

```python
import math

import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node
from sensor_msgs.msg import LaserScan


class ObstacleStop(Node):
    def __init__(self):
        super().__init__('obstacle_stop')
        self.declare_parameter('stop_distance', 0.6)
        self.stop_distance = self.get_parameter('stop_distance').value
        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)
        self.subscription = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10
        )

    def scan_callback(self, scan):
        # Use the center portion of the scan as the front region.
        center = len(scan.ranges) // 2
        start = max(0, center - 20)
        end = min(len(scan.ranges), center + 20)
        front_ranges = scan.ranges[start:end]

        valid = [
            value for value in front_ranges
            if math.isfinite(value)
            and scan.range_min ⩽ value ⩽ scan.range_max
        ]

        command = Twist()
        nearest = min(valid) if valid else float('inf')

        if nearest < self.stop_distance:
            self.get_logger().warn(f'Obstacle at {nearest:.2f} m: STOP')
            command.linear.x = 0.0
            command.angular.z = 0.0
        else:
            command.linear.x = 0.2
            self.get_logger().info(f'Path clear: {nearest:.2f} m')

        self.publisher.publish(command)
```

```
def main(args=None):
    rclpy.init(args=args)
    node = ObstacleStop()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Run only in simulation first:

```
ros2 run ros2_projects obstacle_stop --ros-args -p stop_distance:=0.6
```

Project 8 — URDF and RViz robot

ROS concepts: URDF, link, joint, TF2, robot state publisher, RViz.

Project 8 explanation — robot description and visualization nodes

Single-node version: run `robot_state_publisher` with the URDF and inspect the model in RViz. The learner observes how one description creates a TF tree.

Multi-node version: `robot_state_publisher` publishes frame transforms, `joint_state_publisher` publishes joint values, and RViz subscribes to the robot model and TF data. These are separate nodes with separate responsibilities.

What to understand: URDF describes links and joints, TF2 describes their frame relationships, and RViz visualizes the resulting data.

Project 8 setup

Create the description directory and place the URDF here:

```
mkdir -p ~/ros2_ws/src/ros2_projects/description
```

Place `simple_robot.urdf` in `description/` and `project8_rviz.launch.py` in `launch/`. Install both directories through `setup.py`. Add `robot_state_publisher` and `rviz2` as runtime dependencies.

URDF file: `description/simple_robot.urdf`

```xml
<?xml version="1.0"?>
<robot name="simple_robot">
  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.6 0.4 0.2"/>
      </geometry>
      <material name="blue">
        <color rgba="0.1 0.3 0.8 1.0"/>
      </material>
    </visual>
  </link>

  <link name="sensor_link">
    <visual>
      <geometry>
        <cylinder radius="0.08" length="0.15"/>
      </geometry>
      <material name="red">
        <color rgba="0.8 0.1 0.1 1.0"/>
      </material>
    </visual>
  </link>

  <joint name="sensor_joint" type="fixed">
    <parent link="base_link"/>
    <child link="sensor_link"/>
    <origin xyz="0 0 0.18" rpy="0 0 0"/>
  </joint>
</robot>
```

Launch file: `launch/project8_rviz.launch.py`

```python
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command, PathJoinSubstitution
from launch_ros.substitutions import FindPackageShare


def generate_launch_description():
    urdf = PathJoinSubstitution([
        FindPackageShare('ros2_projects'),
        'description',
        'simple_robot.urdf',
    ])

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{
                'robot_description': Command(['cat ', urdf]),
            }],
            output='screen',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            output='screen',
        ),
    ])
```

Install the `description` directory in `setup.py` before launching.

Project 9 — Gazebo velocity controller

ROS concepts: simulation, `cmd_vel`, simulated sensors, robot plugins, ROS/Gazebo connection.

Project 9 explanation — simulator, controller, and sensor nodes

Single-node version: run one Python controller that publishes `/cmd_vel` to a simulated robot. The simulator provides the robot and physics.

Multi-node version: the Gazebo simulator, Python controller, sensor bridge, and sensor-monitor node all run independently. The controller publishes commands while sensor nodes publish lidar, camera, or odometry data.

What to understand: Gazebo simulates physics, while ROS 2 nodes provide control and processing. The topics are the connection between simulation and Python.

Project 9 setup

Install and configure a compatible Gazebo ROS 2 simulation. Add `project9_gazebo_controller.py` to the Python package and add:

```
'gazebo_controller = ros2_projects.project9_gazebo_controller:main',
```

The simulation must provide a `/cmd_vel` subscriber. Start the simulation first, then run the controller.

File: `project9_gazebo_controller.py`

```python
import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node


class GazeboController(Node):
    def __init__(self):
        super().__init__('gazebo_controller')
        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)
        self.timer = self.create_timer(0.1, self.publish_command)
        self.elapsed = 0.0

    def publish_command(self):
        command = Twist()
        self.elapsed += 0.1

        # Drive forward for 5 seconds, then stop.
        if self.elapsed < 5.0:
            command.linear.x = 0.2
        else:
            command.linear.x = 0.0
            self.timer.cancel()
            self.get_logger().info('Gazebo test motion complete')

        self.publisher.publish(command)


def main(args=None):
    rclpy.init(args=args)
    node = GazeboController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Start a compatible Gazebo ROS robot, then run:

```
ros2 run ros2_projects gazebo_controller
```

Project 10 — Odometry monitor

ROS concepts: odometry, coordinate frames, pose, timestamps, TF2.

Project 10 explanation — odometry reader and TF2 monitor

Single-node version: `odom_monitor` subscribes to `/odom` and prints x, y, and yaw.

Multi-node version: an odometry publisher provides `/odom`, a TF broadcaster provides transforms, and a Python monitor subscribes to odometry or queries TF2. RViz displays the result.

What to understand: odometry is an estimate of movement, while TF2 describes the relationship between frames such as `odom`, `base_link`, and `laser`.

Project 10 setup

Add `project10_odom_monitor.py` and this entry to `setup.py`:

```
'odom_monitor = ros2_projects.project10_odom_monitor:main',
```

The robot or simulator must publish `nav_msgs/msg/Odometry` on `/odom`.

File: `project10_odom_monitor.py`

```python
import math

import rclpy
from nav_msgs.msg import Odometry
from rclpy.node import Node


class OdomMonitor(Node):
    def __init__(self):
        super().__init__('odom_monitor')
        self.subscription = self.create_subscription(
            Odometry, '/odom', self.odom_callback, 10
        )

    def odom_callback(self, message):
        position = message.pose.pose.position
        orientation = message.pose.pose.orientation

        # Convert the quaternion's yaw component approximately for learni
        yaw = math.atan2(
            2.0 * (orientation.w * orientation.z),
            1.0 - 2.0 * (orientation.z * orientation.z),
        )

        self.get_logger().info(
            f'x={position.x:.2f}, y={position.y:.2f}, yaw={yaw:.2f} rad'
        )


def main(args=None):
    rclpy.init(args=args)
    node = OdomMonitor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

Run with a simulated robot publishing `/odom`:

```
ros2 run ros2_projects odom_monitor
```

Project 11 — Python action server and client

ROS concepts: action, goal, feedback, result, cancellation.

Project 11 explanation — action server and client

Single-node version: one node simulates a task internally and reports progress. This teaches the task state machine.

Multi-node version: the action server executes the movement task, while the action client sends a goal, receives feedback, receives the result, and can request cancellation. This is the normal ROS 2 action design.

What to understand: actions are used for tasks that take time. They have a goal, feedback, result, and cancellation path.

Project 11 setup

Create `action/MoveDistance.action` with the definition below. Add ROS action generation to the package, build the generated interface, then add:

```
ros2_projects/ros2_projects/project11_action_server.py
```

Add `rclpy` and the generated action package dependencies. The action server code must be built after the action interface is generated.

Action definition: `action/MoveDistance.action`

```
float32 distance
---
float32 distance_moved
---
bool success
```

Action server: `project11_action_server.py`

```python
import time

import rclpy
from rclpy.action import ActionServer
from rclpy.node import Node
from ros2_projects.action import MoveDistance


class MoveDistanceServer(Node):
    def __init__(self):
        super().__init__('move_distance_server')
        self.server = ActionServer(
            self,
            MoveDistance,
            '/move_distance',
            self.execute_callback,
        )

    def execute_callback(self, goal_handle):
        target = goal_handle.request.distance
        feedback = MoveDistance.Feedback()
        moved = 0.0

        while moved < target:
            if goal_handle.is_cancel_requested:
                goal_handle.canceled()
                result = MoveDistance.Result()
                result.success = False
                return result

            time.sleep(0.1)
            moved += 0.1
            feedback.distance_moved = min(moved, target)
            goal_handle.publish_feedback(feedback)

        goal_handle.succeed()
        result = MoveDistance.Result()
        result.success = True
        return result


def main(args=None):
    rclpy.init(args=args)
```

```
    node = MoveDistanceServer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

The action interface must be generated before building the Python package. A production robot controller should use a ROS timer or non-blocking control loop instead of `time.sleep()`.

Project 12 — Rosbag recorder and analyzer

ROS concepts: recording, replay, timestamps, offline analysis.

Project 12 explanation — recorder and analyzer nodes

Single-node version: use the ROS 2 command-line recorder and replay commands without writing a node. This teaches rosbag basics.

Multi-node version: a robot publishes sensor data, `ros2 bag record` stores it, and a Python analyzer subscribes while the bag is replayed. This allows repeatable offline testing.

What to understand: rosbag records messages with timing so a failure can be replayed and analyzed later.

Project 12 setup

No additional Python package is required for recording. First start a simulator publishing `/tf`, `/odom`, `/scan`, and `/cmd_vel`. Add `project12_bag_analyzer.py` to the Python package and add:

```
'bag_analyzer = ros2_projects.project12_bag_analyzer:main',
```

Record data:

```
ros2 bag record /tf /odom /scan /cmd_vel -o robot_run
```

Inspect it:

```
ros2 bag info robot_run
```

Replay it:

```
ros2 bag play robot_run
```

Python analyzer: `project12_bag_analyzer.py`

```python
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan


class BagAnalyzer(Node):
    def __init__(self):
        super().__init__('bag_analyzer')
        self.warning_count = 0
        self.subscription = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10
        )
        self.timer = self.create_timer(5.0, self.report)

    def scan_callback(self, message):
        valid = [value for value in message.ranges if value == value]
        if valid and min(valid) < 0.5:
            self.warning_count += 1

    def report(self):
        self.get_logger().info(
            f'Obstacle warnings observed: {self.warning_count}'
        )


def main(args=None):
    rclpy.init(args=args)
    node = BagAnalyzer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Run the analyzer while replaying the bag.

Project 13 — Navigation goal client

ROS concepts: localization, map, costmap, planner, navigation action.

Project 13 explanation — navigation client and navigation stack

Single-node version: the Python navigation client sends one goal to an already-running Nav2 system.

Multi-node version: the navigation client, planner, controller, localization, map server, costmaps, and robot driver are separate nodes. The client sends a goal and the navigation stack coordinates the robot.

What to understand: navigation is not one large Python script. It is a group of nodes connected through topics, services, and actions.

Project 13 setup

Install Nav2 and its Python commander package for ROS 2 Jazzy. Add `project13_navigation_client.py` to the package and add:

```
'navigation_client = ros2_projects.project13_navigation_client:main',
```

A compatible simulated robot, map, localization system, and active Nav2 stack are required before running the client.

File: `project13_navigation_client.py`

```python
import rclpy
from geometry_msgs.msg import PoseStamped
from nav2_simple_commander.robot_navigator import BasicNavigator


def main():
    rclpy.init()
    navigator = BasicNavigator()

    navigator.waitUntilNav2Active()

    goal = PoseStamped()
    goal.header.frame_id = 'map'
    goal.header.stamp = navigator.get_clock().now().to_msg()
    goal.pose.position.x = 1.0
    goal.pose.position.y = 0.5
    goal.pose.orientation.w = 1.0

    navigator.goToPose(goal)

    while not navigator.isNavComplete():
        feedback = navigator.getFeedback()
        if feedback is not None:
            print(f'Distance remaining: {feedback.distance_remaining:.2f}

    result = navigator.getResult()
    print(f'Navigation result: {result}')
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

This requires Nav2 and a running simulated robot with a map and localization. Test with simulation first.

Project 14 — Final autonomous robot controller

ROS concepts: integration, safety, topics, parameters, actions, sensors, launch, testing.

Project 14 explanation — final integrated robot

Single-node version: the capstone safety controller reads `/scan`, publishes `/cmd_vel`, and publishes status. This is a small integrated demonstration.

Multi-node version: split the capstone into sensor processing, safety control, status reporting, navigation client, battery monitor, and launch/bringup nodes. This is the recommended production-style architecture.

What to understand: ROS 2 is valuable because independent nodes can be developed, tested, replaced, and reused without rewriting the entire robot system.

Project 14 setup

Add `project14_capstone_safety.py` to the Python package and add:

```
'capstone_safety_controller = ros2_projects.project14_capstone_safety:mai
```

The final system needs a simulator publishing `/scan` and accepting `/cmd_vel`. Configure `stop_distance` in a YAML file and start the complete system with a launch file.

File: `project14_capstone_safety.py`

```python
import math

import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from std_msgs.msg import String


class CapstoneSafetyController(Node):
    def __init__(self):
        super().__init__('capstone_safety_controller')
        self.declare_parameter('stop_distance', 0.6)
        self.stop_distance = self.get_parameter('stop_distance').value

        self.velocity_publisher = self.create_publisher(Twist, '/cmd_vel'
        self.status_publisher = self.create_publisher(
            String, '/robot_status', 10
        )
        self.subscription = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10
        )
        self.last_safe = False

    def scan_callback(self, scan):
        valid = [
            value for value in scan.ranges
            if math.isfinite(value)
            and scan.range_min ⩽ value ⩽ scan.range_max
        ]
        nearest = min(valid) if valid else float('inf')

        command = Twist()
        status = String()

        if nearest < self.stop_distance:
            command.linear.x = 0.0
            status.data = 'STOPPED: obstacle detected'
        else:
            command.linear.x = 0.15
            status.data = 'MOVING: path clear'

        self.velocity_publisher.publish(command)
```

```
        self.status_publisher.publish(status)
        self.get_logger().info(status.data)


def main(args=None):
    rclpy.init(args=args)
    node = CapstoneSafetyController()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        stop = Twist()
        node.velocity_publisher.publish(stop)
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Start this only after a simulator is publishing `/scan` and accepting `/cmd_vel`:

```
ros2 run ros2_projects capstone_safety_controller --ros-args \
  -p stop_distance:=0.6
```

Capstone integration checklist

```
[ ] WSL and ROS 2 installed
[ ] Python package builds
[ ] Publisher and subscriber work
[ ] Service works
[ ] Parameters work
[ ] Launch file works
[ ] Sensor topic is visible
[ ] Safety stop is tested in simulation
[ ] RViz shows the robot and frames
[ ] Gazebo robot moves
[ ] Action or navigation goal works
[ ] Rosbag is recorded
[ ] README explains every command
```

Important note

Projects 1–5 can run with only basic ROS 2 packages. Projects 6–14 require additional packages and, for Gazebo or Nav2, a compatible robot simulation. The code is intentionally educational: review the concept and test in simulation before using any velocity controller with real hardware.

Final project completion rule

For every project, the learner must be able to build the package, run it from a clean Ubuntu terminal, explain the ROS concepts, inspect the graph, change one behavior, and document the result.